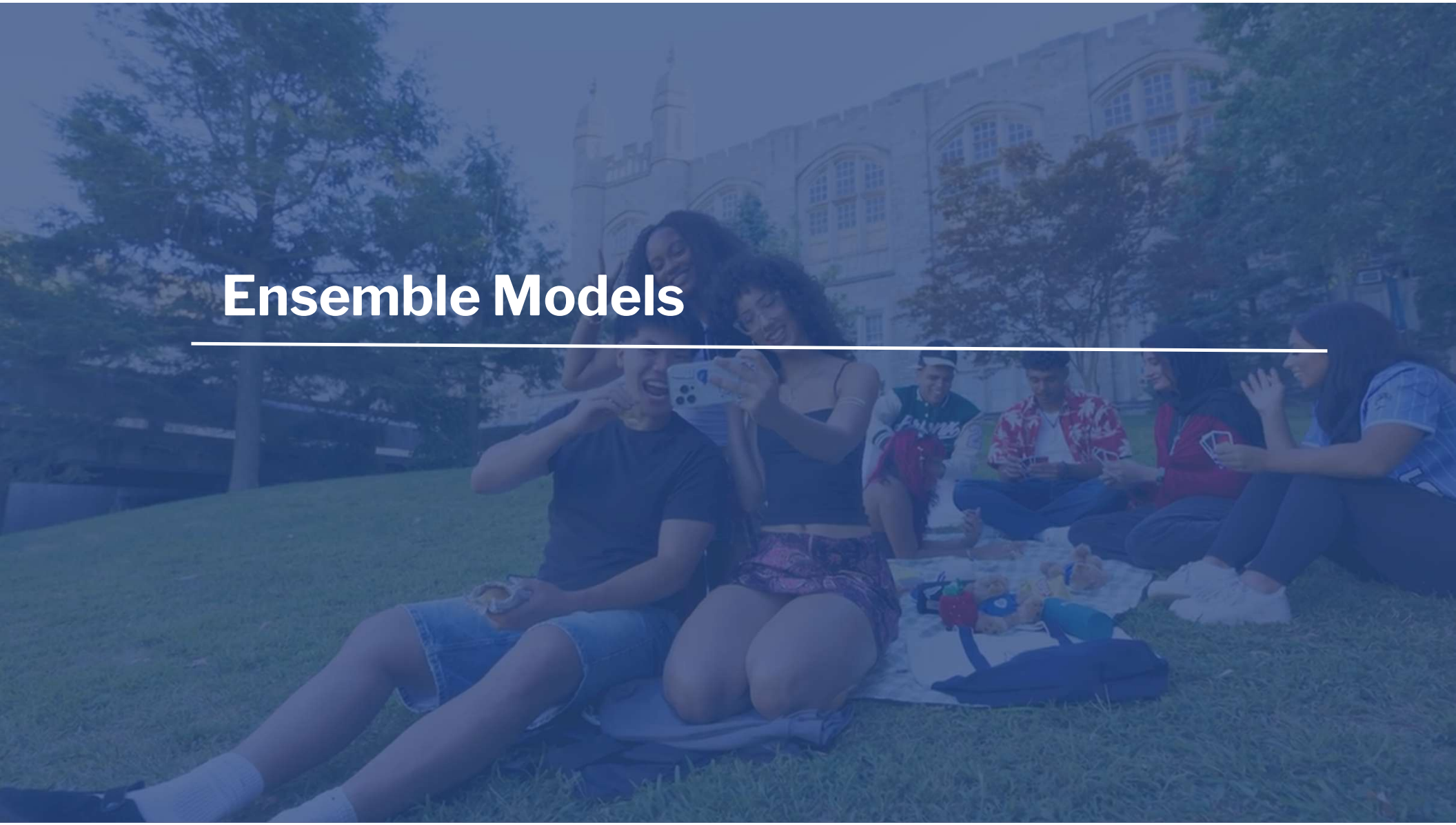
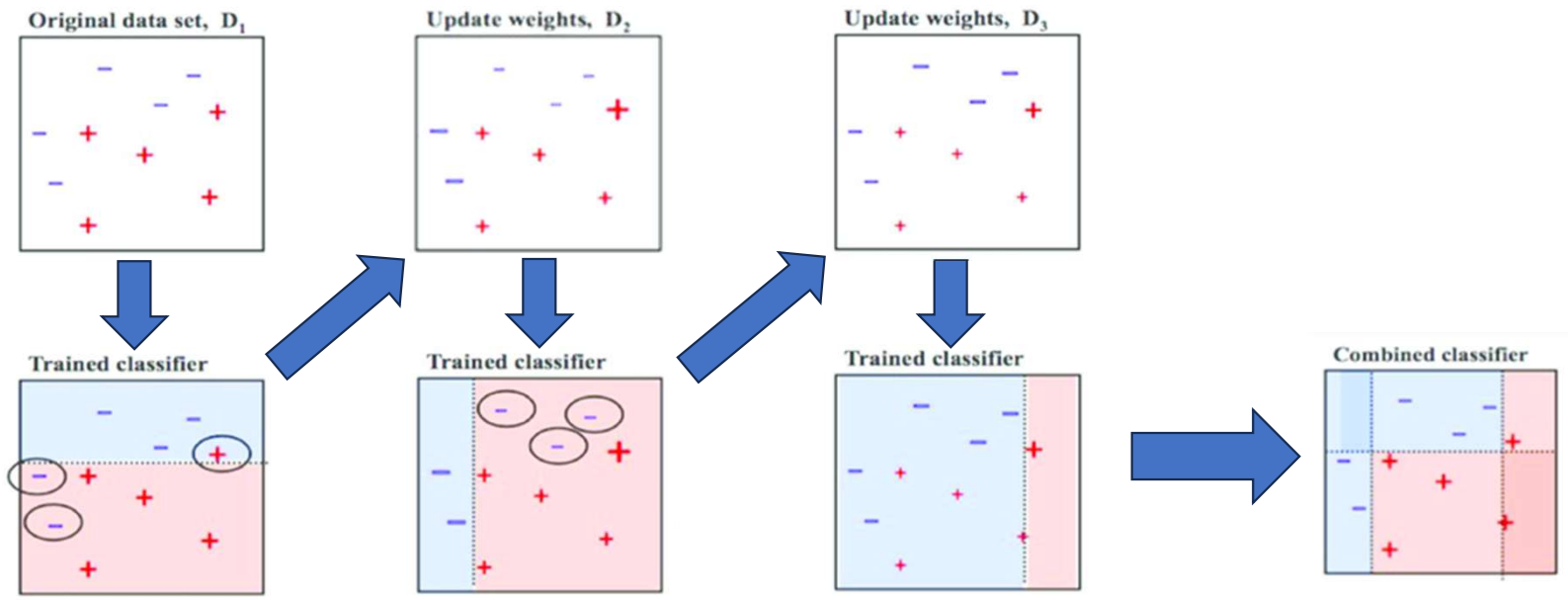

Introduction to Machine Learning
(GAI 601)
WEEK 7
CU
NY
SPS



Ensemble Models

Adaboost (Adaptive Boosting)

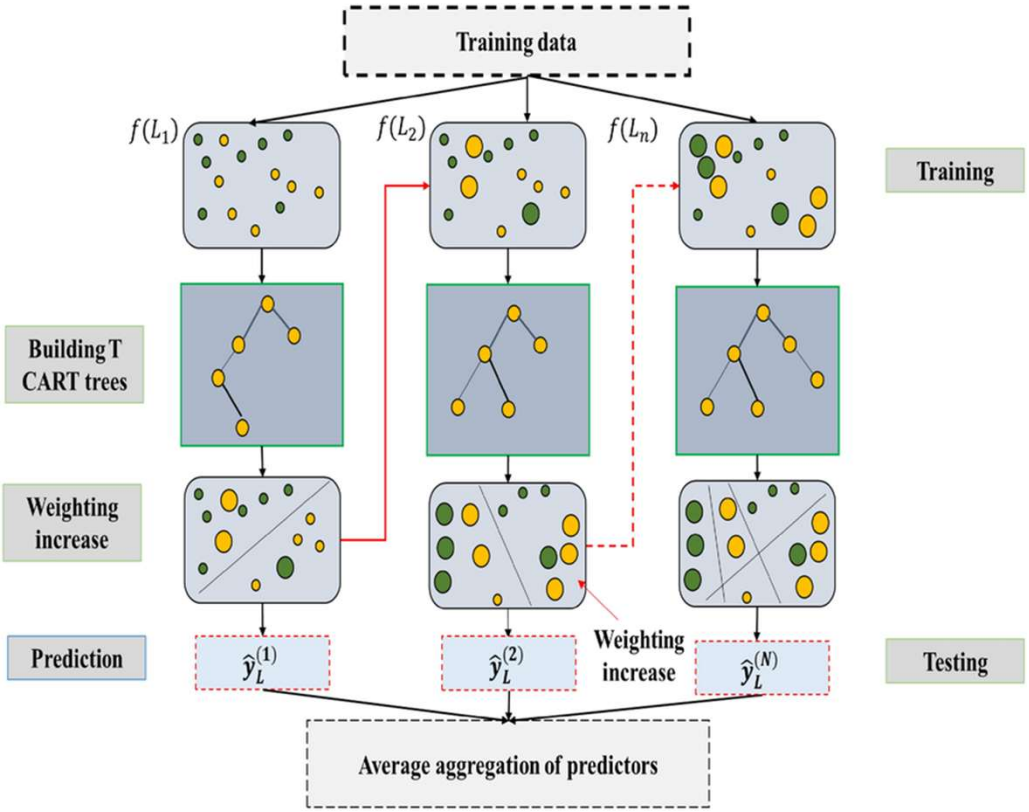
- AdaBoost is the essential boosting algorithm. It reweights the dataset before each new subsampling based on the performance of the last classifier.
- Main difference with bagging: SEQUENTIAL.



Adaboost

- Uses Weak learners (also called base-learners):
  - We are going to train a sequence of weak classifiers (Weak because not as strong as the final classifier)
  - We are not limited to decision trees for the base learners – we could also use neural nets, SVMs, etc.
  - In practice, simple classification trees or stumps as base-learners, which results in improved performance compared to the classification by one tree or a single base-learner.
- Process:
  - The training data sampling will have equal weights, initially.
  - At each step, we use the current weights, train a new classifier, and use its performance on the training data to produce new weights for the next step.
  - Adaboost automatically adjusts its weights for the data based on the performance of the current iteration (weights re-computed iteratively).
  - But we keep ALL the weak classifiers, combining the results from all of the weak classifiers.

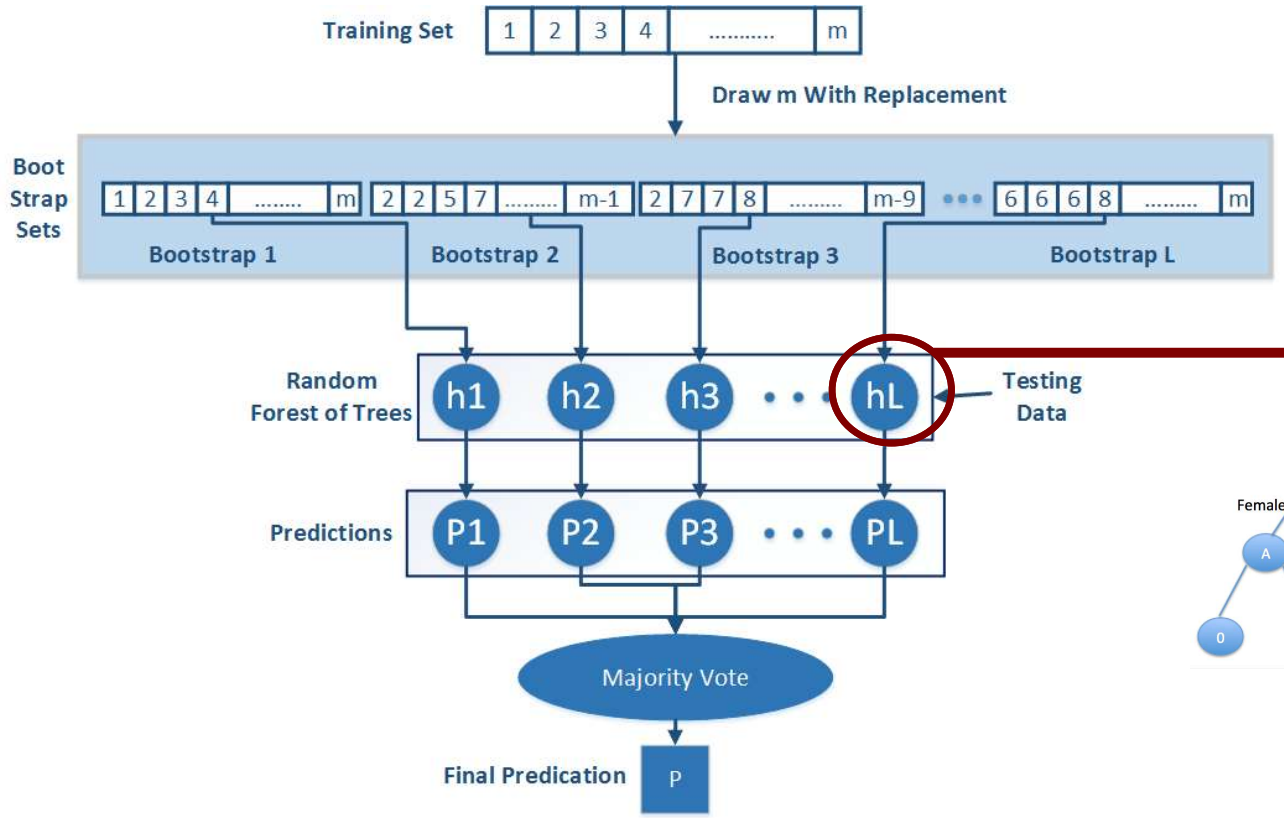
xgBoost


Training data
$f(L_1)$
$f(L_2)$
$f(L_n)$
Training
Building T CART trees
Weighting increase
Prediction
$\hat{y}_L^{(1)}$
$\hat{y}_L^{(2)}$
Weighting increase
$\hat{y}_L^{(N)}$
Testing
Average aggregation of predictors

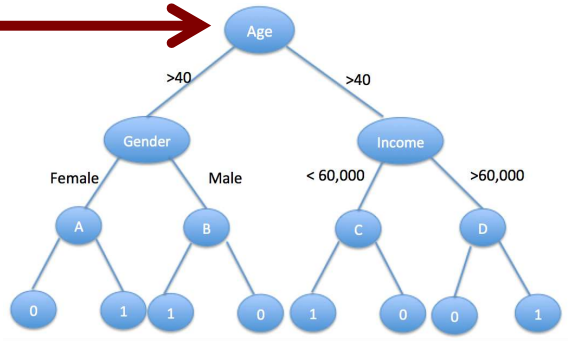
xgBoost

- Gradient Boost is a robust machine learning algorithm made up of **Gradient descent** and **Boosting**.
- Train new base-learners on **residuals** of overall model. Constitutes gradient boosting because approximating the residual and adding to the previous result is essentially a form of gradient descent.
- Gradient Boosting has three main components: additive model, loss function and a weak learner.
- The word ‘gradient’ implies that you can have two or more derivatives of the same function.
- The technique yields a direct interpretation of boosting methods from the perspective of numerical optimization in a function space and generalizes them by allowing optimization of an arbitrary loss function.

Random Forest



A Random Tree



CUNY School of Professional Studies

Random Forest

- Machine learning ensemble algorithm
  - Based on tree models. Every tree is independent
  - Combining multiple predictors
- In addition to sampling datapoints we **also sample features** (to increase independence among classifiers)
- Compute many decision trees and output majority vote
- Can also rank features
- Suitable for both regression and classification
- Automatic variable selection
- Handles missing values well
- Robust, improving model stability and accuracy

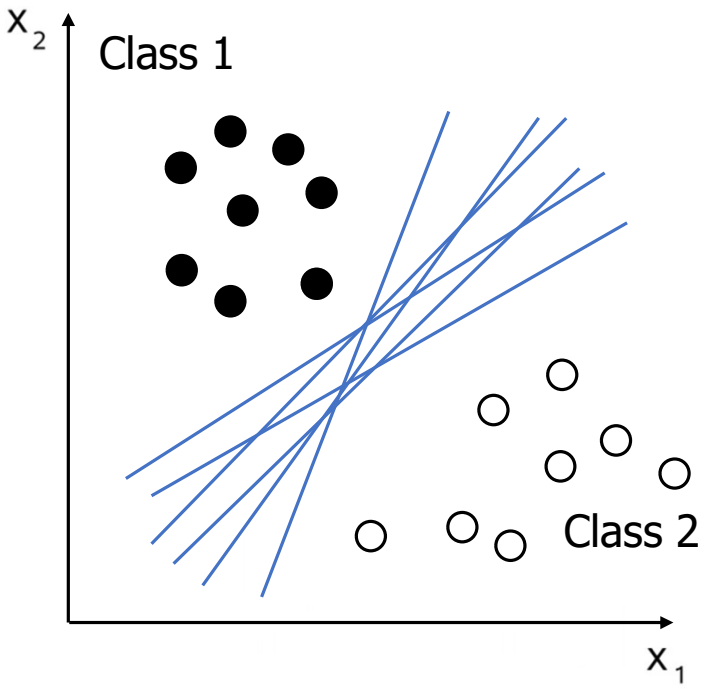
AdaBoost vs Gradient Boost

	AdaBoost	Gradient Boost
Loss Function	Minimises the exponential loss function that can make the algorithm sensitive to the outliers	any differentiable loss function can be utilized. Gradient Boosting algorithm is more robust to outliers than AdaBoost.
Flexibility	Designed boosting algorithm with a particular loss function	Gradient Boosting is more flexible than AdaBoost.
Benefits	Minimizes loss function related to any classification error and is best used with weak learners	Very good predictive accuracy
Shortcomings	Identified by high-weight data points	Shortcomings of the existing weak learners can be identified by gradients
Use-cases	Binary classification problems	Used for both classification and regression problems


Support Vector Machines

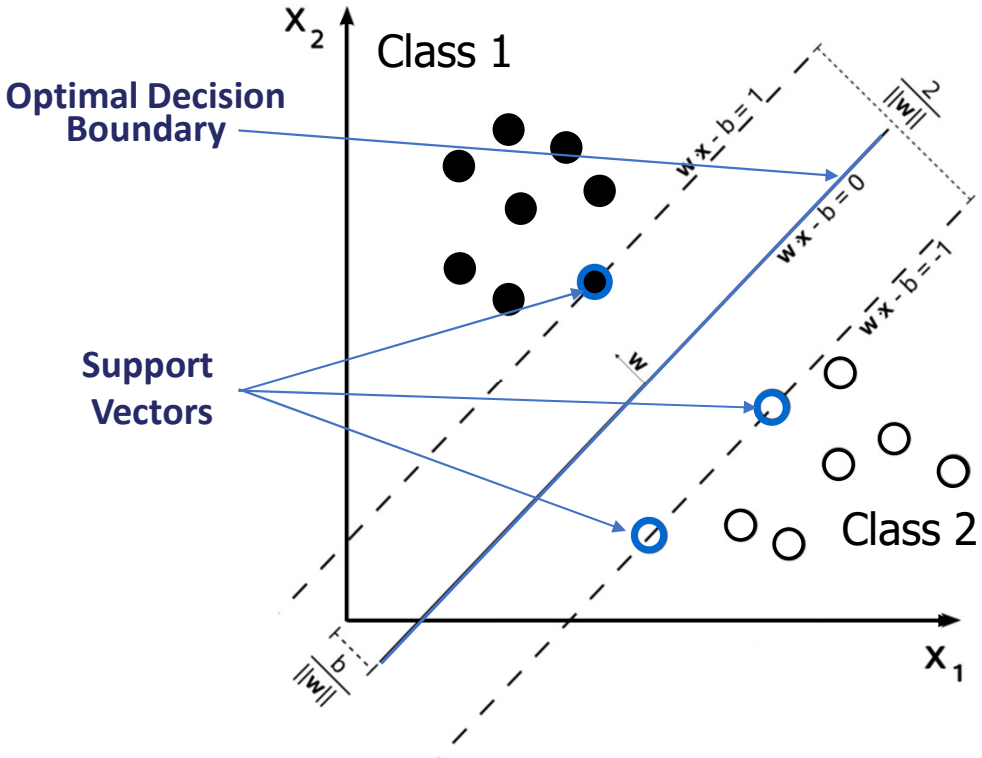
Support Vector Machines

How do you find the right decision boundary?



Support Vector Machines

SVM approach: Maximize the margin, to define the decision boundary



$$\max \frac{2}{\|w\|}$$

subject to

$$y_n \begin{cases} +1 & w^T x + b \geq 1 \\ -1 & w^T x + b \leq -1 \end{cases}$$

which we can write as $y_n(w^T x + b) \geq 1$

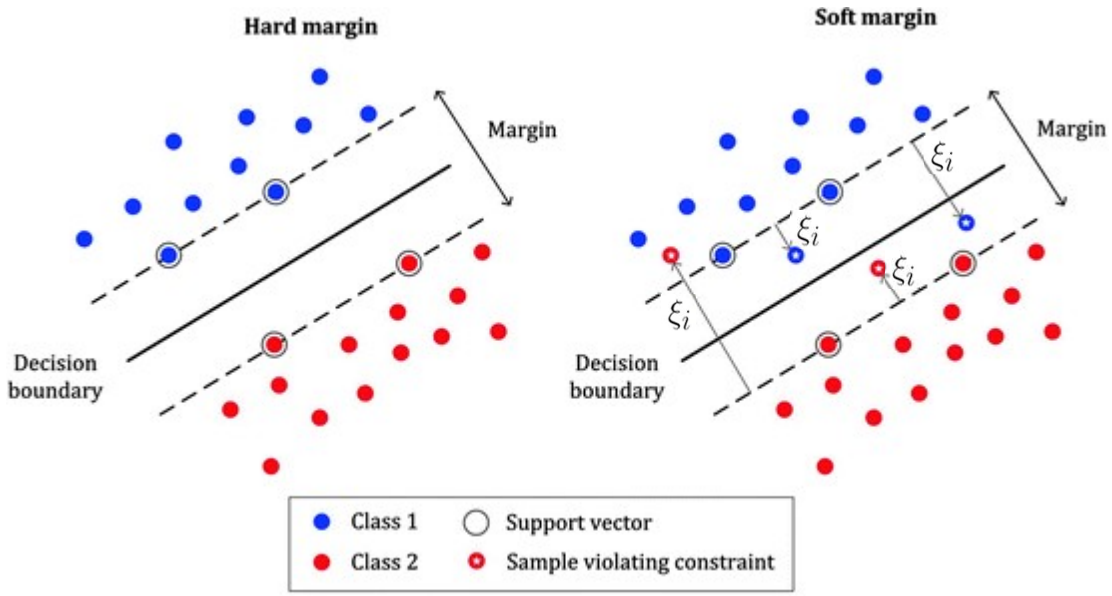
$$max \frac{2}{||w||} = max \frac{1}{||w||} = min||w|| = min \frac{1}{2}||w||^2$$

$$b = \frac{1}{S} \sum_{i=1}^{S} (y_i - w \cdot x)$$

$$w = \sum_{i=1}^{m} \alpha_i y_i x_i$$

Soft-margin SVM

So far we have considered the case with no errors (hard margin).
How do we handle the case where there is is overlap?

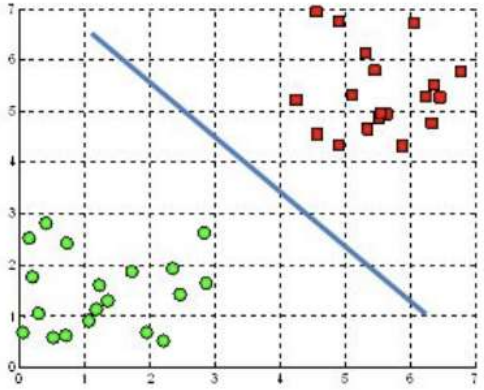


$$\underbrace{\min \frac{1}{2}\|w\|^2}_{\text{regularizer}} + \underbrace{C_i \sum_{i=1}^{n} \xi_i}_{\text{error term}}$$

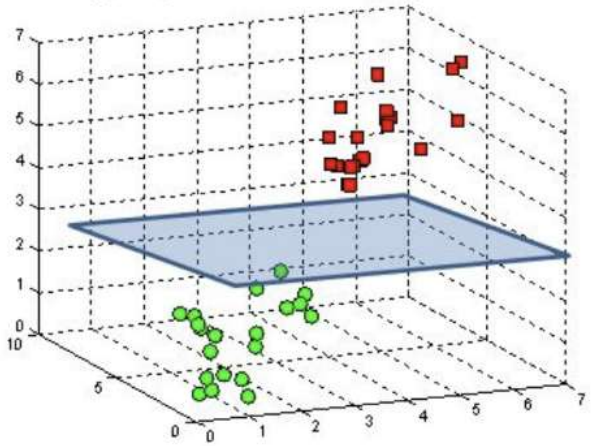
Hyperplanes

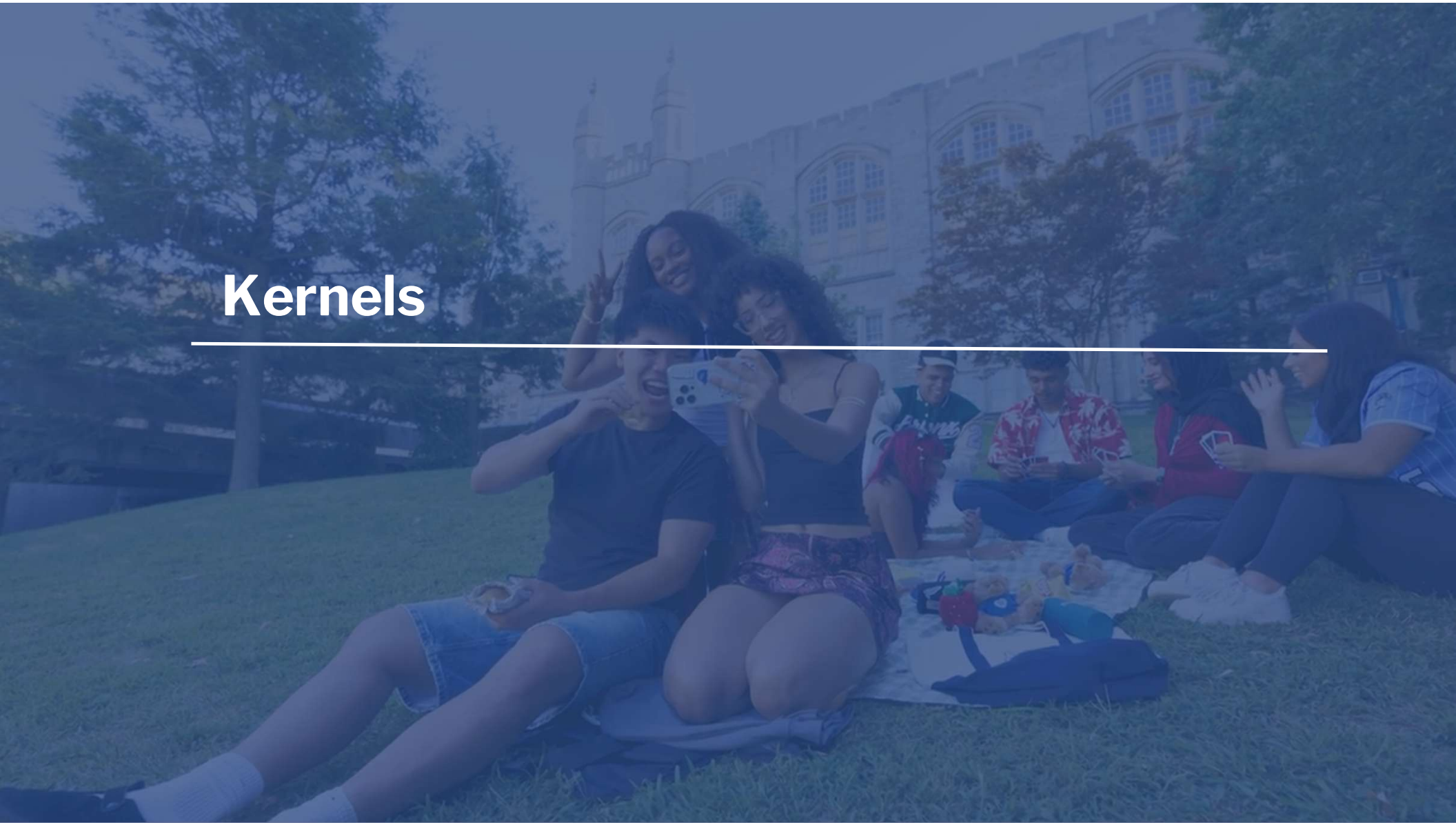
A decision boundary is defined by a hyperplane

A hyperplane in $\mathbb{R}^2$ is a line



A hyperplane in $\mathbb{R}^3$ is a plane

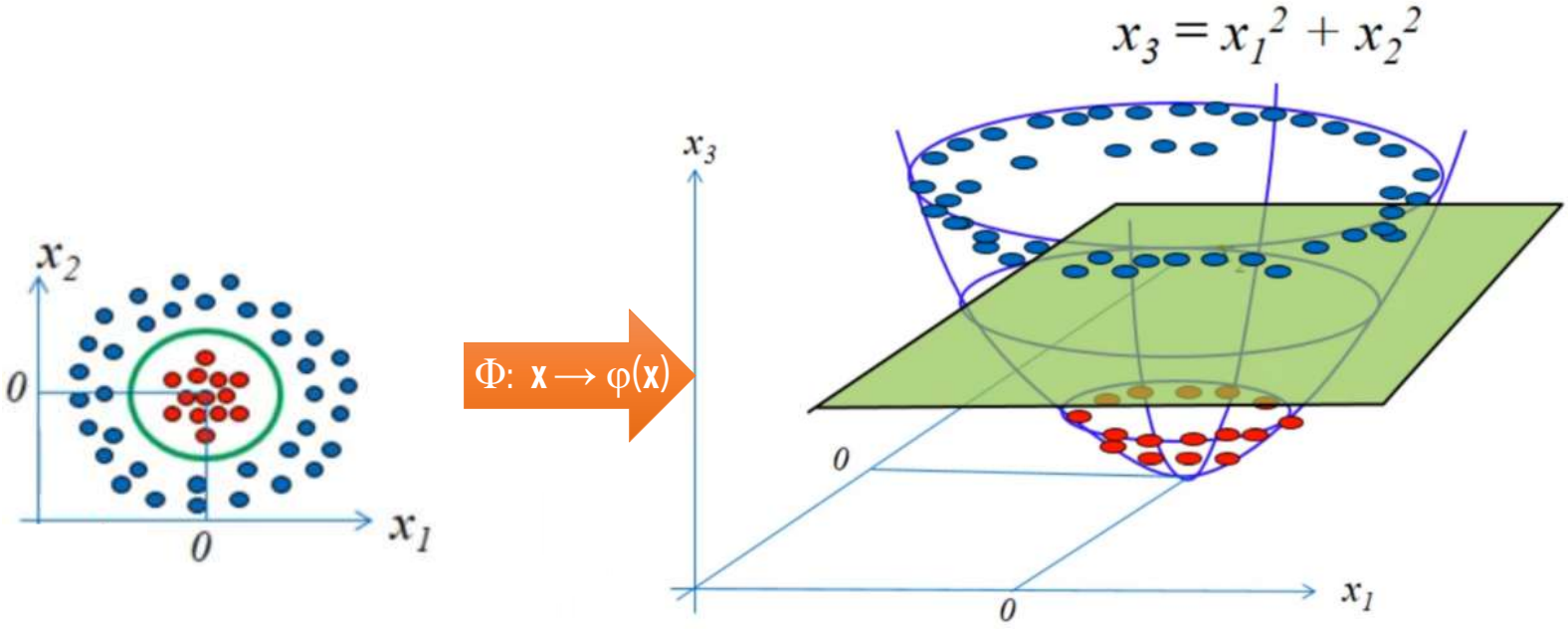




Kernels

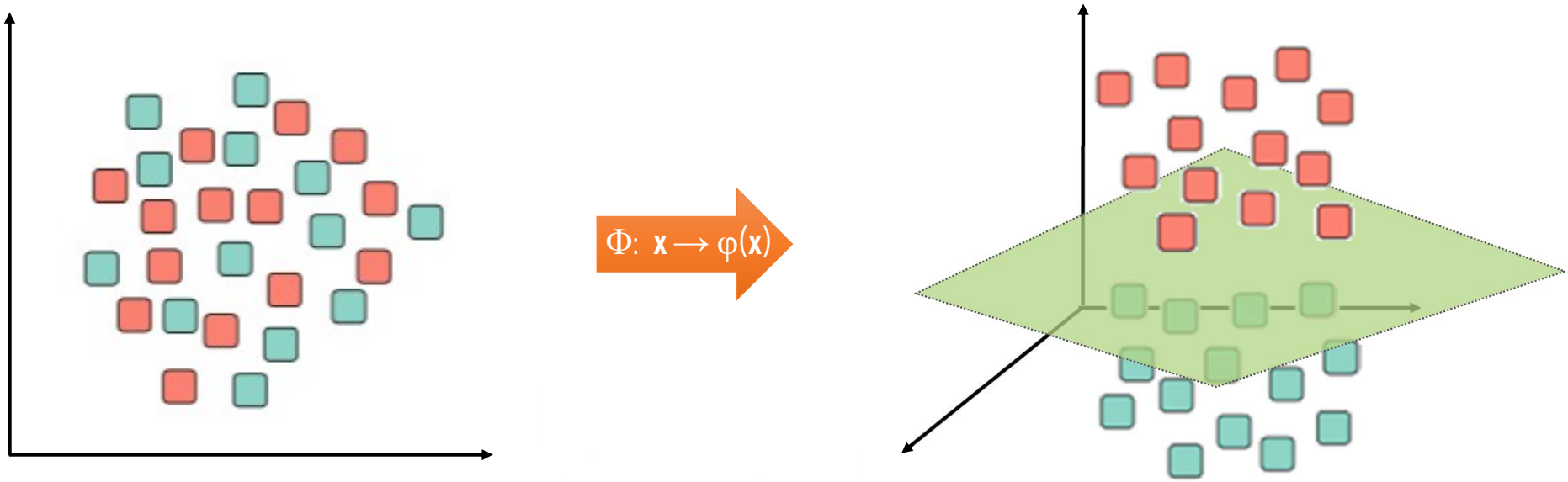
The ‘kernel trick’

How do we handle non-linear decision boundaries?
Map the original input space to a higher-dimensional feature space where the training set is separable.

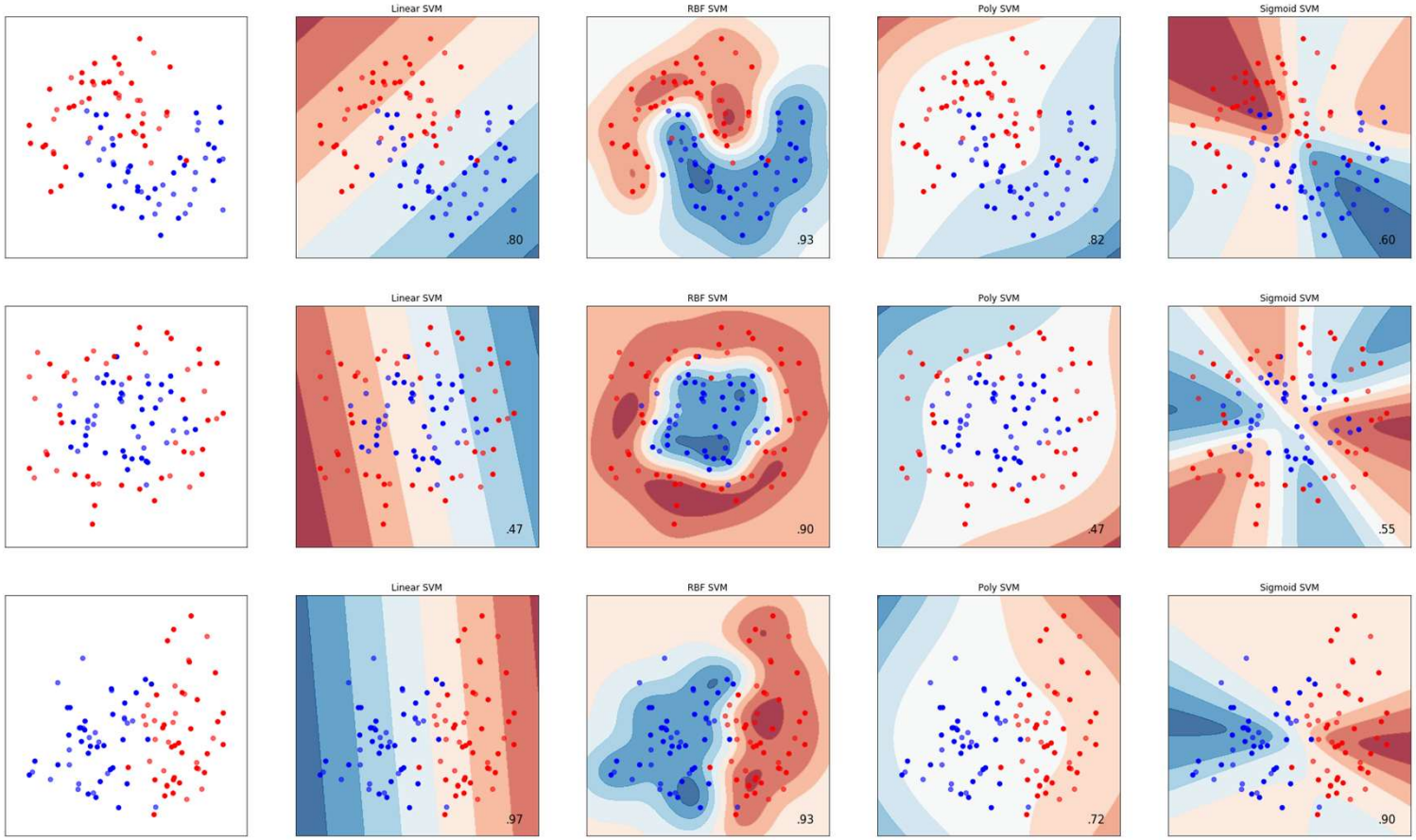


The ‘kernel trick’

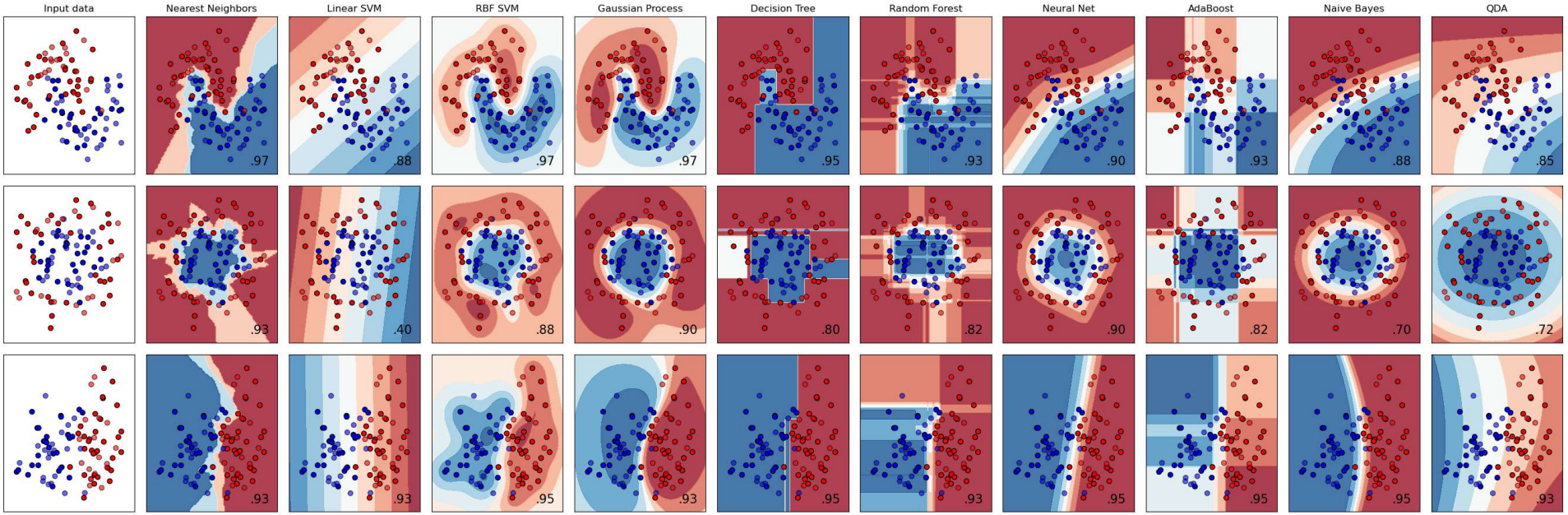
Kernels allow us to keep using a classifier with a linear decision boundary – because we have embedded the data in a higher-dimensional feature space



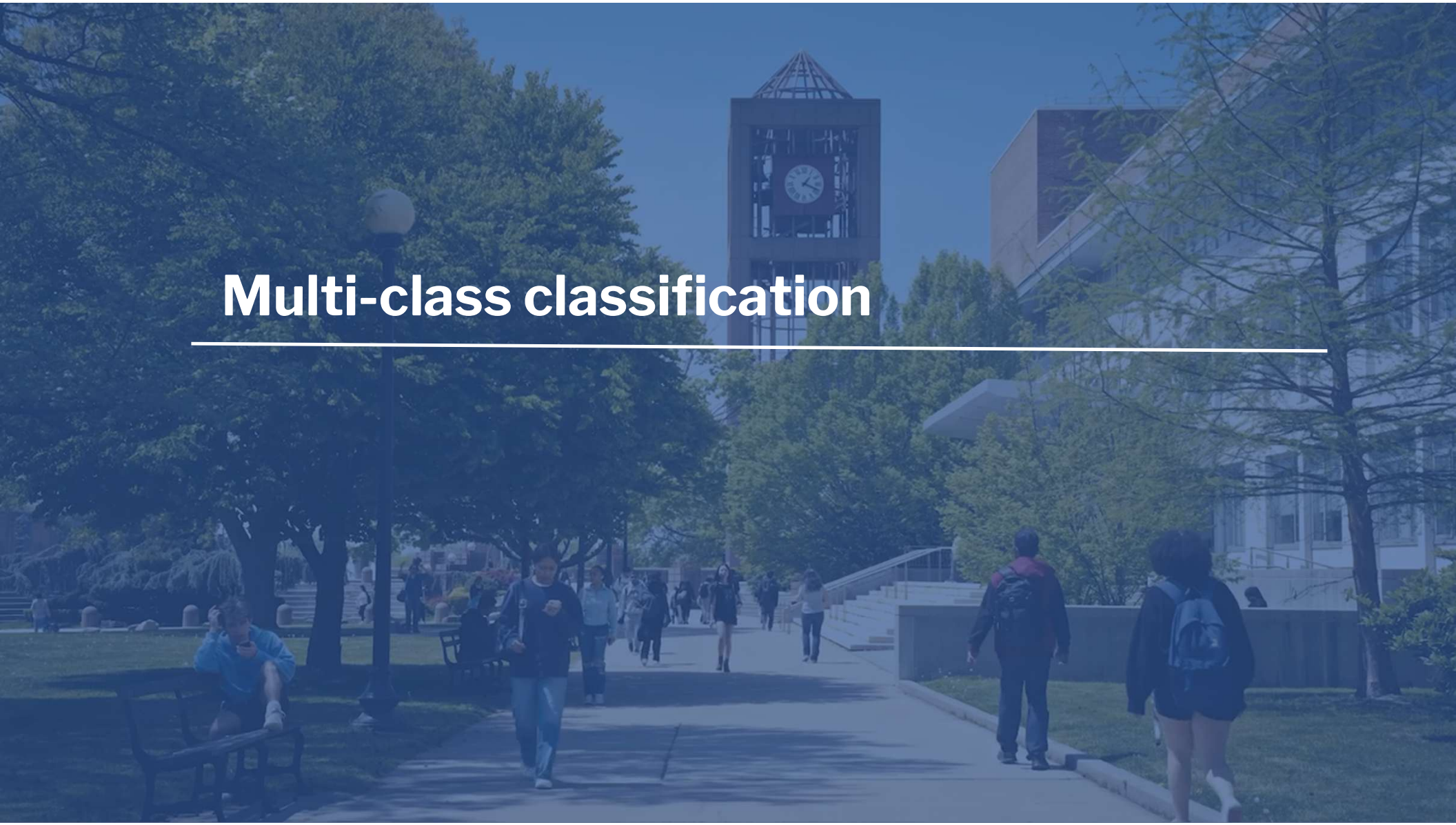
Comparison of kernels



Comparison of kernels

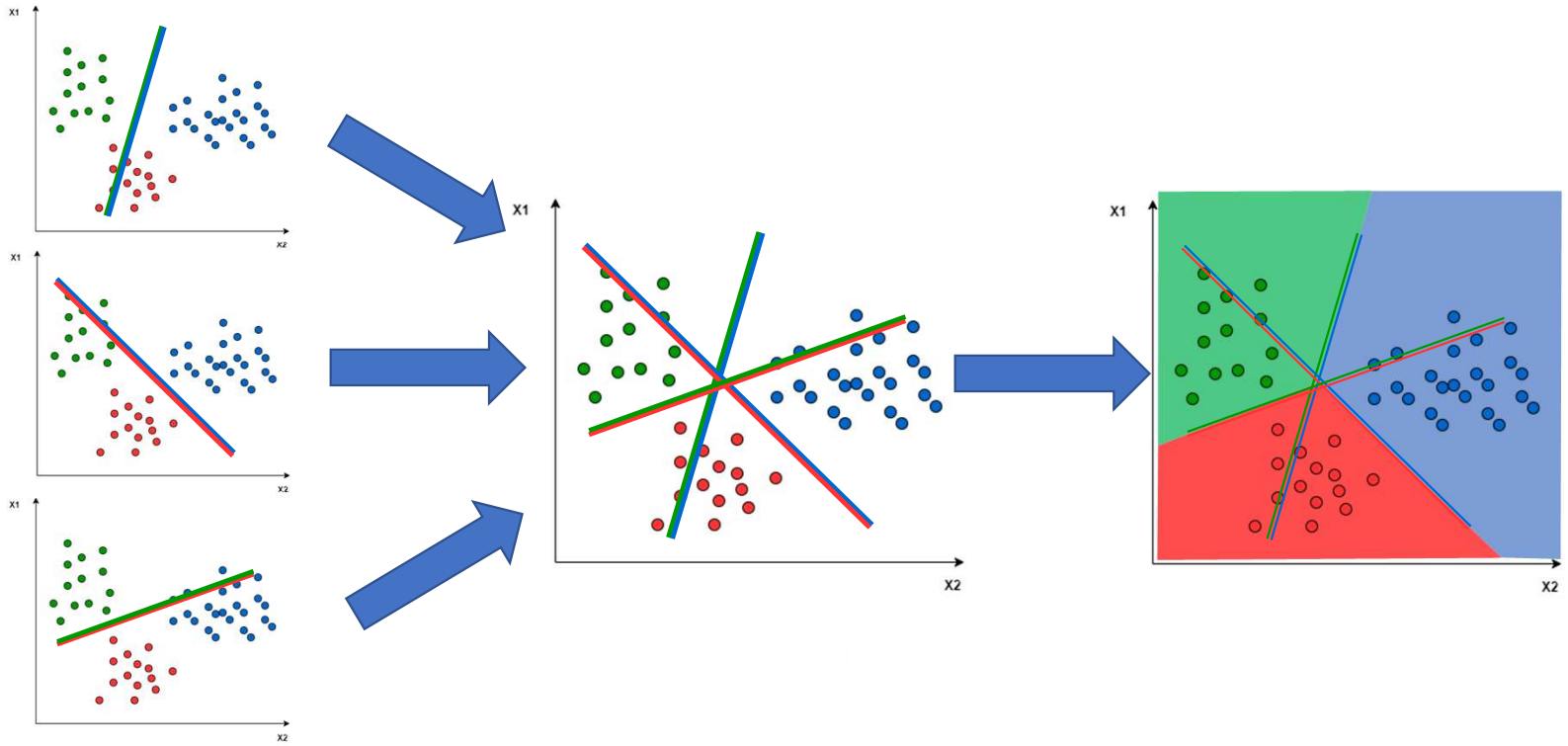


Source: https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html


Multi-class classification

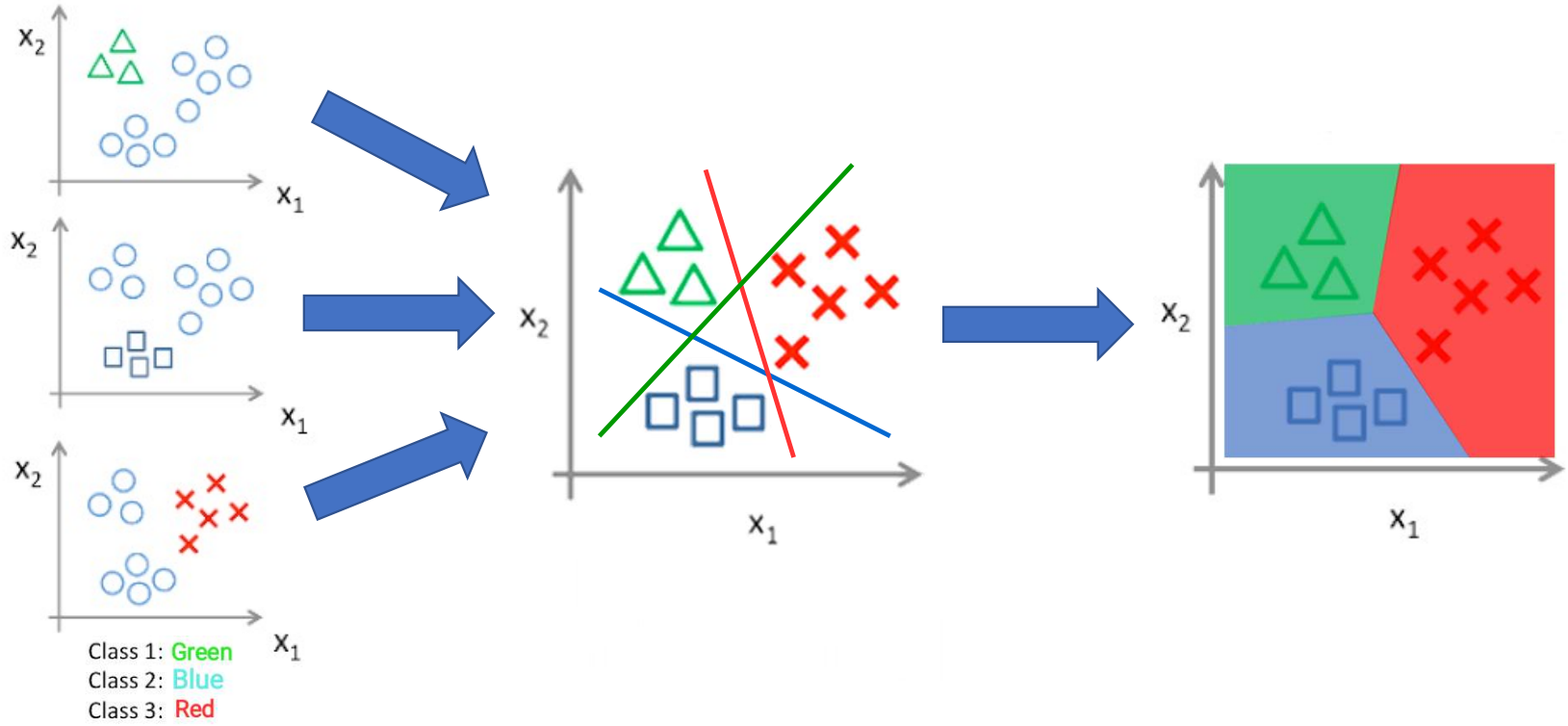
Multi-class SVM: One-vs-one

A hyperplane is generated between every two classes, neglecting the points of the third class.



Multi-class SVM: One-vs-rest

A hyperplane is generated each class and all other classes at once.



Benefits of SVMs

- Works well on smaller cleaner datasets
- It can be more efficient because it uses a subset of training points
- Different kernel functions can be specified for the decision function
- Sparseness of solution when dealing with large data sets
  - Only support vectors are used to specify the separating hyperplane
- Ability to handle large feature spaces
  - Complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- Guaranteed to converge to a single global solution (convex optimization)

Weaknesses of SVMs

- Sensitive to noise
  - A relatively small number of mislabeled examples can dramatically decrease the performance
- Essentially a binary classifier
- Isn't suited to larger datasets as the training time with SVMs can be high
- Less effective on noisier datasets with overlapping classes
- If the number of features is a lot bigger than the number of data points, avoiding over-fitting when choosing kernel functions and regularization term is crucial.


CUNY | School of
Professional Studies